

```

1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5
6 from sklearn.preprocessing import MinMaxScaler
7 from tensorflow.keras.models import Sequential
8 from tensorflow.keras.layers import LSTM, Dense
9
10
11 # 1. Download NIFTY 50 data
12 data = yf.download(
13     "^NSEI",
14     start="2015-01-01",
15     end="2026-01-01"
16 )
17
18 # 2. Select closing price
19 close_data = data[['Close']].values
20
21
22 # 3. Normalize data
23 scaler = MinMaxScaler(feature_range=(0, 1))
24 scaled_data = scaler.fit_transform(close_data)
25
26
27 # 4. Create sequences
28 sequence_length = 60
29
30 X = []
31 y = []
32
33 for i in range(sequence_length, len(scaled_data)):
34     X.append(scaled_data[i-60:i, 0])
35     y.append(scaled_data[i, 0])
36
37 X = np.array(X)
38 y = np.array(y)
39
40
41 # 5. Train-test split
42 train_size = int(len(X) * 0.8)
43
44 X_train = X[:train_size]
45 X_test = X[train_size:]
46
47 y_train = y[:train_size]
48 y_test = y[train_size:]
49
50
51 # 6. Reshape for LSTM
52 X_train = X_train.reshape(
53     X_train.shape[0],
54     X_train.shape[1],
55     1
56 )
57
58 X_test = X_test.reshape(
59     X_test.shape[0],
60     X_test.shape[1],
61     1
62 )
63
64
65 # 7. Build LSTM model
66 model = Sequential([
67     LSTM(
68         50,
69         input_shape=(60, 1)
70     ),
71     Dense(1)
72 ])
73
74
75 # 8. Compile

```

```

76 model.compile(
77     optimizer='adam',
78     loss='mean_squared_error'
79 )
80
81
82 # 9. Train
83 history = model.fit(
84     X_train,
85     y_train,
86     epochs=10,
87     batch_size=32,
88     validation_split=0.1
89 )
90
91
92 # 10. Prediction
93 predictions = model.predict(X_test)
94
95 predictions = scaler.inverse_transform(
96     predictions
97 )
98
99 actual = scaler.inverse_transform(
100     y_test.reshape(-1, 1)
101 )
102
103
104 # 11. Plot results
105 plt.figure(figsize=(12, 6))
106
107 plt.plot(
108     actual,
109     label='Actual'
110 )
111
112 plt.plot(
113     predictions,
114     label='Predicted'
115 )
116
117 plt.title('NIFTY 50 LSTM Forecasting')
118 plt.xlabel('Time')
119 plt.ylabel('NIFTY 50 Closing Price')
120 plt.legend()
121 plt.show()
122
123
124 # 12. Predict next day
125 last_60_days = scaled_data[-60:]
126
127 X_future = last_60_days.reshape(
128     1, 60, 1
129 )
130
131 prediction = model.predict(X_future)
132
133 prediction = scaler.inverse_transform(
134     prediction
135 )
136
137 print(
138     "Predicted Next Day NIFTY 50:",
139     prediction[0][0]
140 )
141

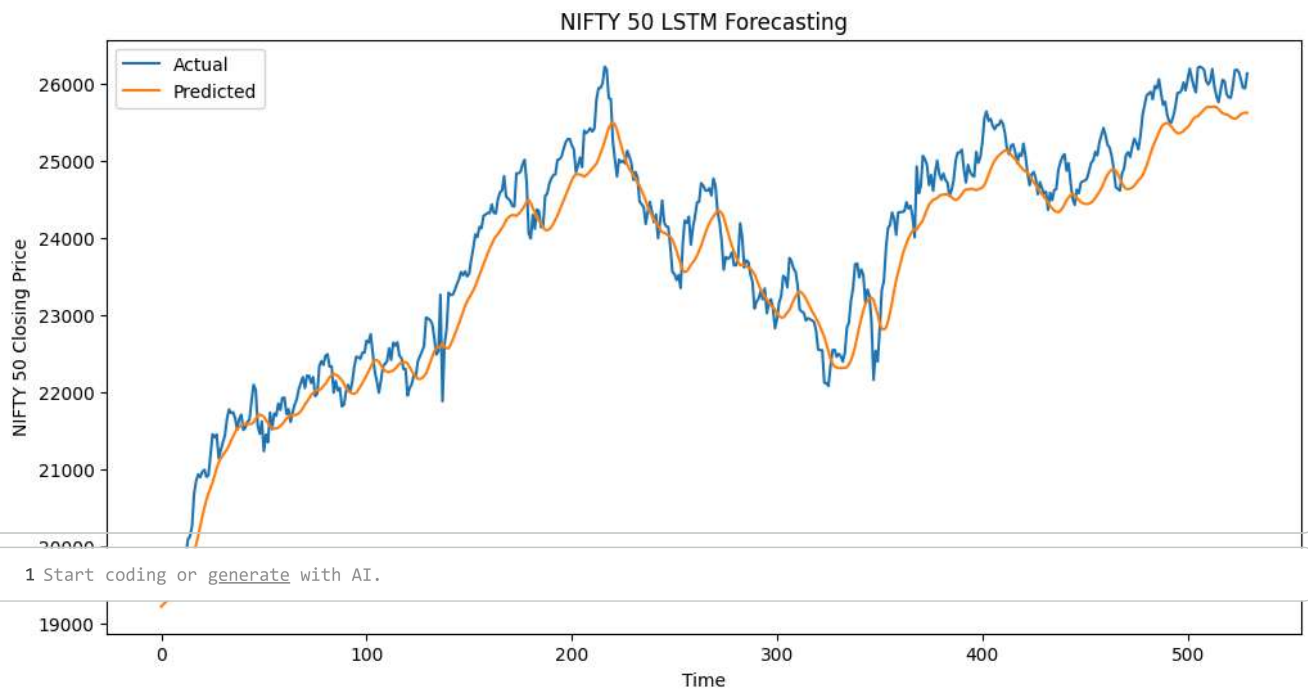
```

```

/tmp/ipykernel_760/2661970709.py:12: FutureWarning: YF.download() has changed argument auto_adjust default to True
data = yf.download(
[*****100%*****] 1 of 1 completedEpoch 1/10

/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` a
super().__init__(**kwargs)
60/60 ————— 3s 26ms/step - loss: 0.0115 - val_loss: 5.3364e-04
Epoch 2/10
60/60 ————— 1s 22ms/step - loss: 3.2348e-04 - val_loss: 3.5335e-04
Epoch 3/10
60/60 ————— 2s 25ms/step - loss: 2.7085e-04 - val_loss: 2.0948e-04
Epoch 4/10
60/60 ————— 2s 34ms/step - loss: 2.4810e-04 - val_loss: 1.8481e-04
Epoch 5/10
60/60 ————— 2s 22ms/step - loss: 2.3574e-04 - val_loss: 2.0821e-04
Epoch 6/10
60/60 ————— 1s 22ms/step - loss: 2.2726e-04 - val_loss: 1.6939e-04
Epoch 7/10
60/60 ————— 1s 22ms/step - loss: 2.1242e-04 - val_loss: 2.0672e-04
Epoch 8/10
60/60 ————— 1s 22ms/step - loss: 2.0636e-04 - val_loss: 1.7962e-04
Epoch 9/10
60/60 ————— 3s 24ms/step - loss: 1.9619e-04 - val_loss: 2.1075e-04
Epoch 10/10
60/60 ————— 2s 32ms/step - loss: 1.9002e-04 - val_loss: 1.4785e-04
17/17 ————— 1s 24ms/step

```



1 Start coding or [generate](#) with AI.

1/1 ————— 0s 59ms/step
 Predicted Next Day NIFTY 50: 25630.824